CSED233 · 데이터구조 · 포스텍

# Ch2. One-Day C++ — 자료구조 구현 토대

## 1. 컴파일 파이프라인

### 이론 & 배경

- **C++**: C 확장, cross-platform, OOP(class), 고성능. 주요 버전 C++11/14/17/20/23.
- **Build 과정**:
  - **Compiler(컴파일러)**: `.cpp/.h` → `.o` (linux) / `.obj` (windows) (machine-code 변환)
  - **Linker(링커)**: 여러 `.o` + library → 실행 파일( `.exe` / linux 바이너리)
- 실습: `g++ -o my_main main.cpp` → `./my_main`

### Q1. Compiler의 입출력은?

> **힌트**

컴파일은 소스 파일 단위로 일어나고, 결과물은 기계어 오브젝트 파일이다.

> **정답**

개념은 힌트 참고.

**In**: `.cpp` , `.h` / **Out**: `.o` , `.obj` (번역 단위 = .cpp 하나당 하나; .h는 #include로 포함)

### Q2. Linker의 입출력은?

> **힌트**

링커는 여러 오브젝트 파일을 하나의 실행 파일로 묶는다.

> **정답**

개념은 힌트 참고.

**In**: 다수의 `.o` / **Out**: `.exe` (Windows) / 확장자 없는 실행 파일(Linux)

## 2. 변수 · 타입 · Namespace(네임스페이스)

### 기본 타입

`bool`, `char`, `short`, `int`, `long long`, `float` (f32), `double` (f64).

- **Namespace** `std::` : `cout`, `cin`, `endl` 등 표준 라이브러리 식별자 scope.
- `using std::string;` 으로 prefix 생략 가능.

### Float64 vs Float32

- **f64 사용**: 정밀도 중요(과학/금융 계산, 반올림 오차를 줄일 뿐 제거하진 않음)
- **f32 사용**: 메모리/속도 중요, GPU, 이미지 처리 등 precision 덜 필요

### Q3. 직접 만든 `void cout(...)` 와 `std::cout` 을 동시에 사용하려면?

> **힌트**

이름이 같아도 namespace가 다르면 다른 함수다. 전역 공간의 이름은 `::이름` 으로 명시할 수 있다.

> **정답**

개념은 힌트 참고.

`::cout(s)` (global) vs `std::cout << s` (namespaced)로 구분해 호출한다.

## 3. Pointer(포인터), Reference(레퍼런스), Array (DS 구현 핵심)

### Pointer(포인터)

- 모든 변수는 메모리 주소를 가짐 → **pointer**는 그 주소를 저장하는 변수.
- 타입 `T*` = T 변수에 대한 포인터.
- 연산자:
  - `&x` : x의 주소 (address-of)
  - `*q` : q가 가리키는 값 (dereference)

```
char ch = 'Q';
char* p = &ch;   // p holds address of ch
```

```
cout << *p;     // 'Q'
*p = 'X';       // ch 값이 'X'로 변경
```

### Array(배열) ↔ Pointer(포인터) 동등성

- `T[N]` : 같은 타입 N개 연속 저장.
- **배열명 = 첫 원소 pointer**: 대부분의 식에서 그렇게 변환 (sizeof, 단항 &는 예외 — 배열 자체 취급)

```
char c[] = {'c','a','t'};
char* p = c;       // p → c[0]
char* q = &c[0];   // q → c[0]
cout << c[2] << p[2] << q[2]; // "ttt"
```

- 초기화 시 크기 생략 가능: `int a[] = {10,11,12,13};`

## 4. Dynamic Memory(동적 메모리) (new/delete)

### Heap(힙) 할당

- **Free store(프리 스토어) / heap**: runtime에 object를 동적 할당.
- `new T` → T 크기 할당 + 포인터 리턴
- `delete p` → 해제
- **자동 GC 없음**: 할당한 것은 반드시 `delete`. 안 하면 **memory leak**.

### 배열 동적 할당

```
char* buffer = new char[500];
buffer[3] = 'a';
delete[] buffer;  // 배열은 delete[] 필수
```

- 핵심: 일반 객체는 `delete`, 배열은 `delete[]`. 섞으면 UB.

## 5. Reference (별칭)

### 참조

- `T&` : 기존 객체의 **alias**. 선언 시 반드시 초기화, NULL 불가.

- Pointer와 차이: reference는 재할당 불가, dereference 불필요.

```
string author = "Samuel Clemens";
string& penName = author;  // alias
penName = "Mark Twain";    // author도 바뀜
```

### 함수 인자 전달

```
void f(int value, int& ref) {
  value++;  // 원본 영향 없음 (pass by value)
  ref++;    // 원본 수정 (pass by reference)
}
```

- **DS 구현에서 중요**: 크기 큰 객체 복사 비용 회피, 원본 수정이 필요할 때.

## 6. 제어 흐름 & 함수

### For loop

```
for (init; condition; increment) { body }
```

- `break`: innermost loop/switch 즉시 탈출
- `continue`: 현재 iteration 나머지 skip

### Function

- **Declaration** (prototype): 타입 + 이름 + 인자 리스트
- **Definition**: declaration + body
- **Overloading(오버로딩)**: 같은 이름, 다른 인자 리스트 → 목적은 같으나 타입이 다른 경우

```
void print(int x);
void print(const Passenger& p);  // overloaded
```

## 7. Struct(구조체) vs Class(클래스)

### 차이

- **Struct**: 데이터 그룹화 중심. 멤버 기본 **public**.
- **Class**: 데이터 추상화/재사용 중심. 멤버 기본 **private**.
- 문법적 차이는 default access modifier뿐.

### String (STL)

```
#include <string>
using std::string;
string s = "to be";
string t = "not" + s;
int i = s.size();  // 4
char c = s[3];     // 'e'
s += " Smith";     // 추가
```

- Lexicographic 비교는 ASCII 기반 좌→우.

## 8. Casting(캐스팅)

### 명시적 캐스팅

```
int cat = 14;
double dog = (double)cat;  // C-style
double pig = double(cat);  // functional
```

### 함정

```
int i1=18, i2=16;
double dv1 = i1/i2;                 // 1.0 (int 나눗셈 먼저!)
double dv2 = double(i1)/double(i2);  // 1.125
double dv3 = double(i1/i2);         // 1.0 (이미 int)
```

- **핵심**: 나눗셈 **전**에 캐스팅해야 함. 괄호 위치 주의.

## Summary

- Compiler/Linker
- Variable, pointer, array, dynamic memory, function, class/struct, string
- Header file, namespace, cin/cout, enumeration, reference
- Casting, Loop, break/continue